Computer Organization

Final Project

CPU Design

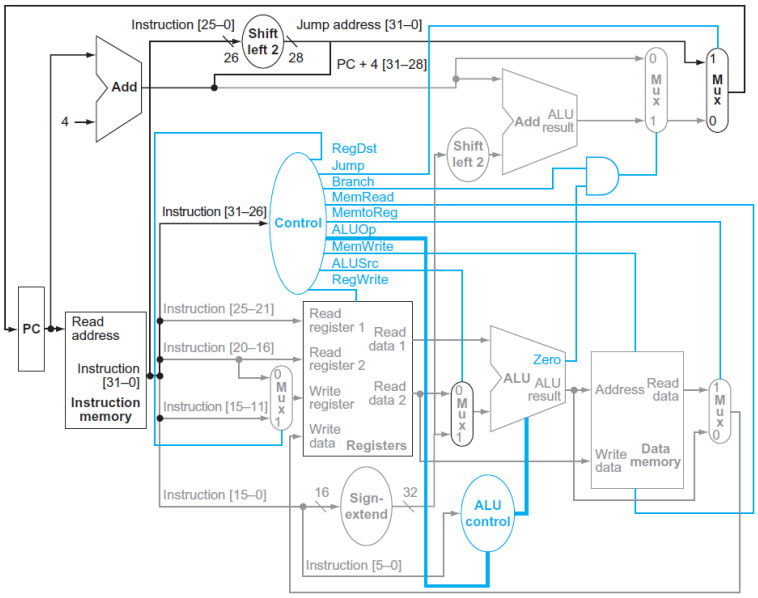
Professor：Tsung-Han Tsai

Member

EE3B	Sena
EE3B	Zuri
EE3B	Nori
EE3B	Nova

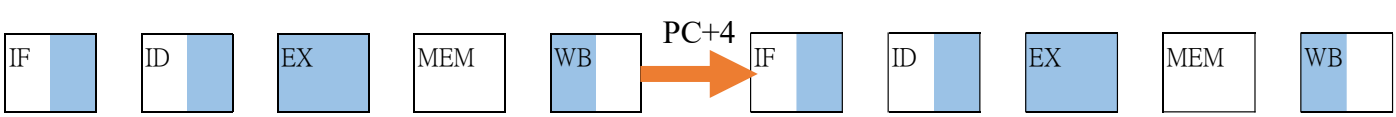
一、 Single cycle implementation

1. 架構



2. 架構說明

我們是先從 pipeline implementation 開始做，所以 pipeline 做出來後，single cycle implementation 相對來說就變得簡單，我們只需要將原本一個 cycle 給一個指令改為等到 IF、ID、EX、MEM、WB 都做完後再給下一個指令即可。因此，我們仍存在五個 stage，只是 clk cycle 會變成五倍。



3. I/O of stages

Stage	Definition	I/O	Parameter	Explanation
IF	計算下一個 pc 的值，並抓取對應的 instruction	input	branch	判斷是否 branch
			pc	初始的 program counter
			jump	判斷是否 jump
			rst	初始化
			clk	時脈
		output	newpc	下個指令位置
			ins_IF	依據 newpc 產生的指令
ID	解碼 instruction，並對 I-type 的後 16bit 做 sign extend	input	ins_IF	依據 newpc 產生的指令
			zero	暫存器(from WB to ID)
			clk	時脈
			rst	初始化
		output	ins_ID	傳遞至 ID 的 instruction，

				為指令的第二個 stage
			sign_extend	將 I-type 最後的 16bits 補成 32bits
			Read_data1	rs 地址所對應的值
			Read_data2	rt 地址所對應的值
EX	對 I-type、R-type 指令做 ALU 運算	input	newpc	下個指令位置
			Read_data1	rs 地址所對應的值
			Read_data2	rt 地址所對應的值
			ins_ID	ID 輸入至 EX 的 instruction
			sign_extend	將 I-type 最後的 16bits 補成 32bits
			clk	時脈
			rst	初始化
		output	Alu_result	ALU 的運算結果
			ins_EX	傳遞至 EX 的 instruction， 為指令的第三個 stage
			Write_data_mem	欲存入 memory 的值
			branch	判斷是否 branch
			jump	判斷是否 jump
MEM	當指令為 lw、sw 時，將 data 由 memory 讀出或寫入	input	ins_EX	自 EX 輸入 MEM 的 instruction
			Alu_result	ALU 的運算結果
			Write_data_mem	欲存入 memory 的值
			clk	時脈
			rst	初始化
		output	Read_data_mem	自 memory 讀出， 欲寫入 register 的值
			ins_MEM	傳遞至 MEM 的 instruction， 為指令的第四個 stage
			Mem reg	memory
WB	將最後運算結果寫回暫存器	input	ins_MEM	自 MEM 輸入 WB 的 instruction
			Read_data_mem	自 memory 讀出， 欲寫入 register 的值
			clk	時脈
			rst	初始化
		output	zero	暫存器(from WB to ID)

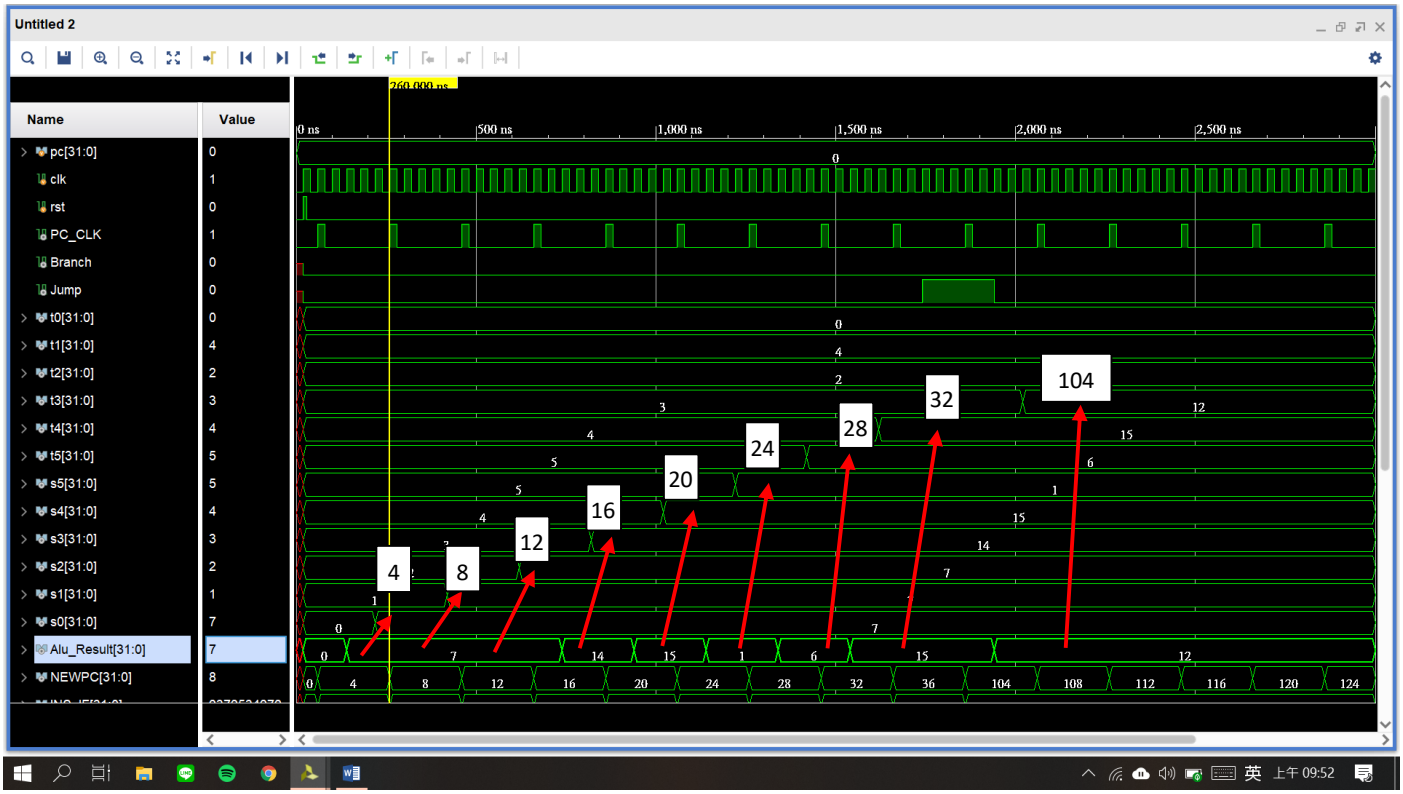
4. Instruction Set

PC	Instruction	Machine Code	Result
4	addi $s0, $t0, 7	001000 01100 01011 00000 00000 000111	s0=7
8	lw $s1, 0($s0)	100011 01011 01010 00000 00000 000000	s1=7
12	lw $s2, 0($s1)	100011 01010 01001 00000 00000 000000	s2=7
16	add $s3, $s1, $s2	000000 01010 01001 01000 00000 100000	s3=14
20	addi $s4, $s1, 8	001000 01010 00111 00000 00000 001000	s4=15
24	sub $s5, $s4, $s3	000000 00111 01000 00110 00000 100010	s5=1
28	and $t5, $s3, $s1	000000 01000 01010 00101 00000 100100	t5=6
32	or $t4, $s3, $s1	000000 01000 01010 00100 00000 100101	t4=15
36	j 104	000010 00000 00000 00000 00000 011010	nextpc=104
104	addi $t3, $s1, 5	001000 01010 00011 00000 00000 000101	t3=12

5. 指令解碼、暫存器及記憶體地址

function code		記憶體	記憶體位址 (十進位)	值	暫存器	程式宣告名稱	地址	初始值
add	100000	mem0	40	2	t0	reg0	01100	0
sub	100010	mem1	36	2	t1	reg1	00001	4
and	100100	mem2	32	2	t2	reg2	00010	2
or	100101	mem3	28	2	t3	reg3	00011	3
slt	101010	mem4	24	2	t4	reg4	00100	4
op code		mem5	20	2	t5	reg5	00101	5
R-type	000000	mem6	16	2	s5	reg6	00110	5
lw	100011	mem7	12	2	s4	reg7	00111	4
sw	101011	mem8	8	2	s3	reg8	01000	3
beq	000100	mem9	4	2	s2	reg9	01001	2
addi	001000	mem10	0	2	s1	reg10	01010	1
					s0	reg11	01011	0
					nop1	reg12	11100	0
					nop2	reg13	11110	0
					zero	zero	11111	0

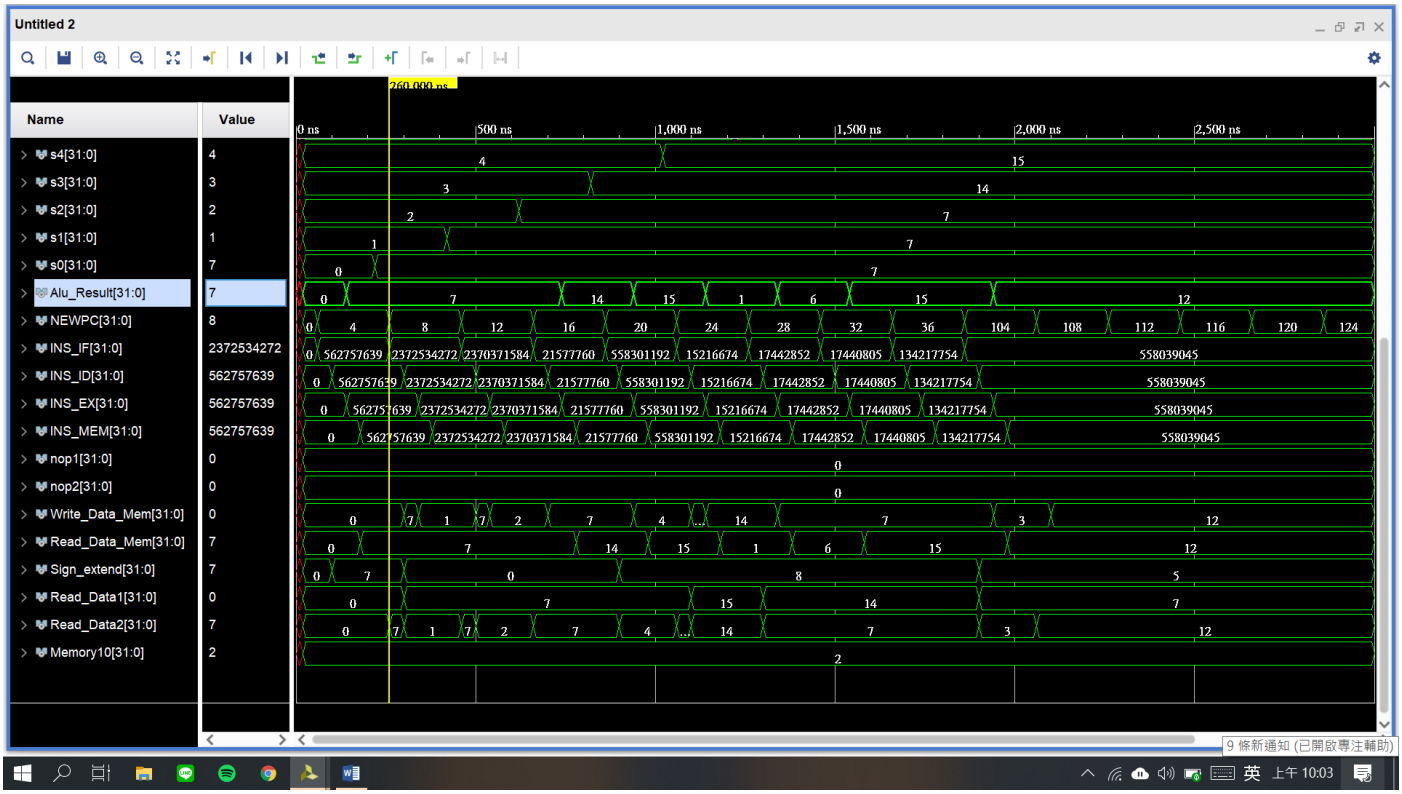
6. 波形圖



PC	Instruction	Result
4	addi $s0, $t0, 7	s0=7
8	lw $s1, 0($s0)	s1=7
12	lw $s2, 0($s1)	s2=7
16	add $s3, $s1, $s2	s3=14
20	addi $s4, $s1, 8	s4=15
24	sub $s5, $s4, $s3	s5=1
28	and $t5, $s3, $s1	t5=6
32	or $t4, $s3, $s1	t4=15
36	j 104	nextpc=104
104	addi $t3, $s1, 5	t3=12

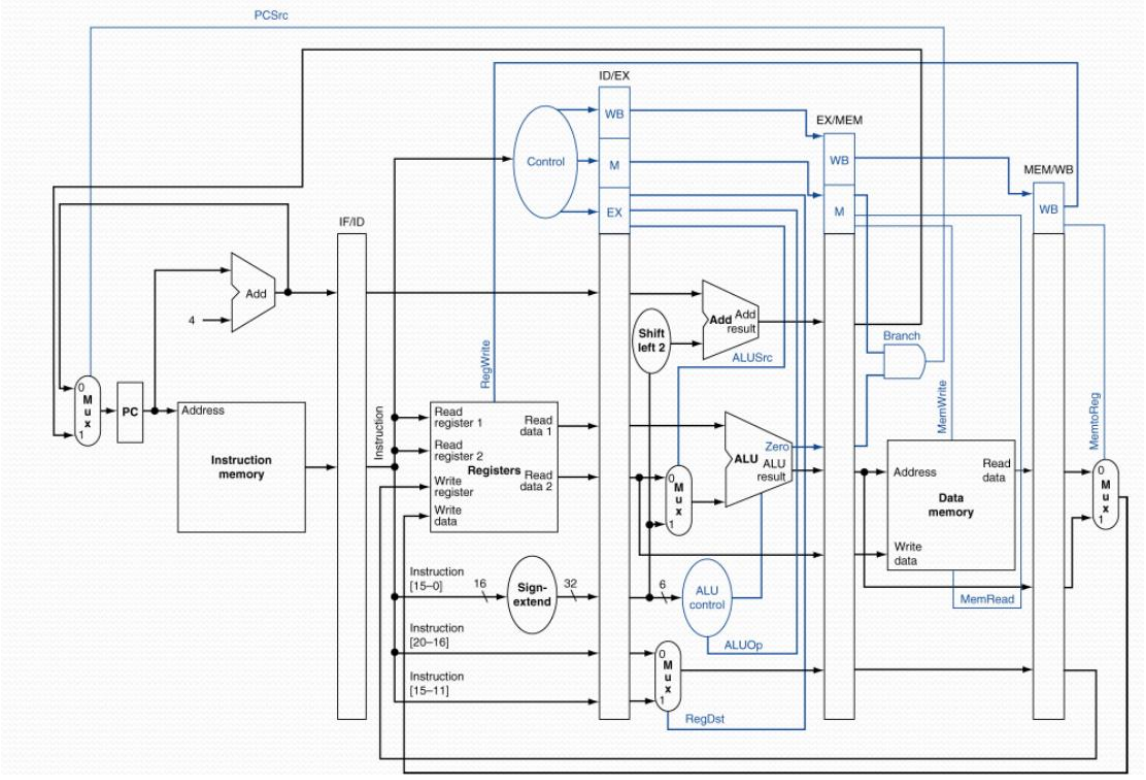
圖中白色方框內代表 newpc 的值，紅色箭頭代表將 newpc 的指令所產生的值寫入到指令指定的暫存器。表左即為相對應的指令及運算結果。

因每經過一個 stage 會經過一個 clk cycle，運算結果會比指令輸入時晚 4 個 clk 才會出現（IF 到 WB 共經過 4 個 CLK）。直到寫完後才會輸入下一個指令（即 PC_Clk 跳起時）。

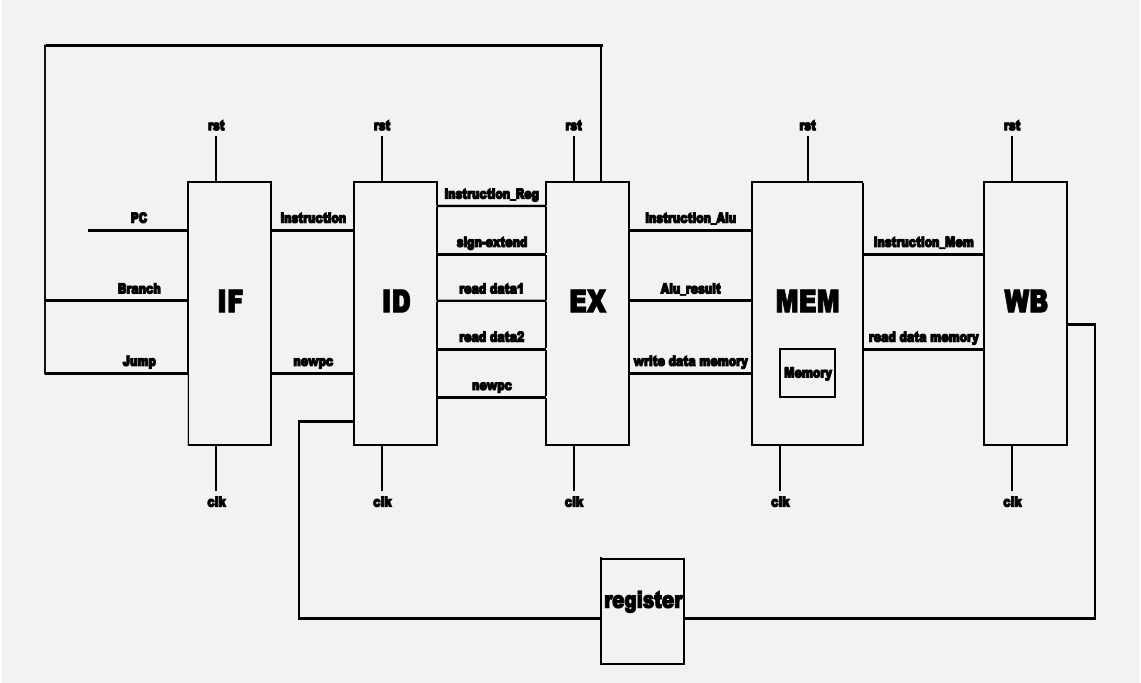

Untitled 2
260.000 ns
Name
Value
0 ns
500 ns
1,000 ns
1,500 ns
2,000 ns
2,500 ns
s4[31:0] 4
s3[31:0] 3
s2[31:0] 2
s1[31:0] 1
s0[31:0] 7
Alu_Result[31:0] 7
NEWPC[31:0] 8
INS_IF[31:0] 2372534272
INS_ID[31:0] 562757639
INS_EX[31:0] 562757639
INS_MEM[31:0] 562757639
nop1[31:0] 0
nop2[31:0] 0
Write_Data_Mem[31:0] 0
Read_Data_Mem[31:0] 7
Sign_extend[31:0] 7
Read_Data1[31:0] 0
Read_Data2[31:0] 7
Memory10[31:0] 2
9 條新通知 (已開啟專注輔助)
英
上午 10:03

二、 Pipeline implementation

1. 架構



2. Block diagram

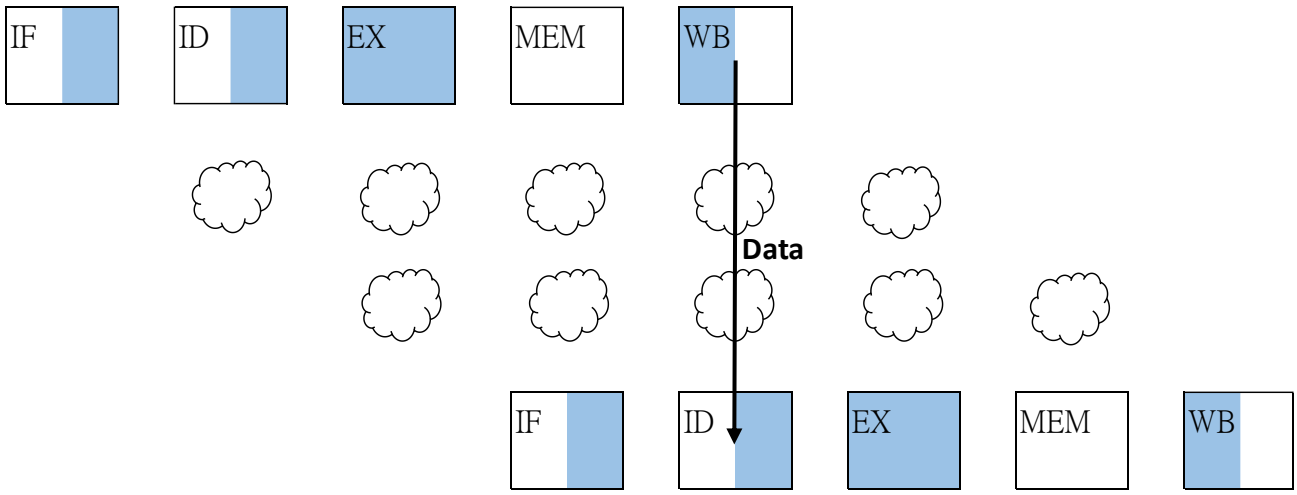


3. 架構說明

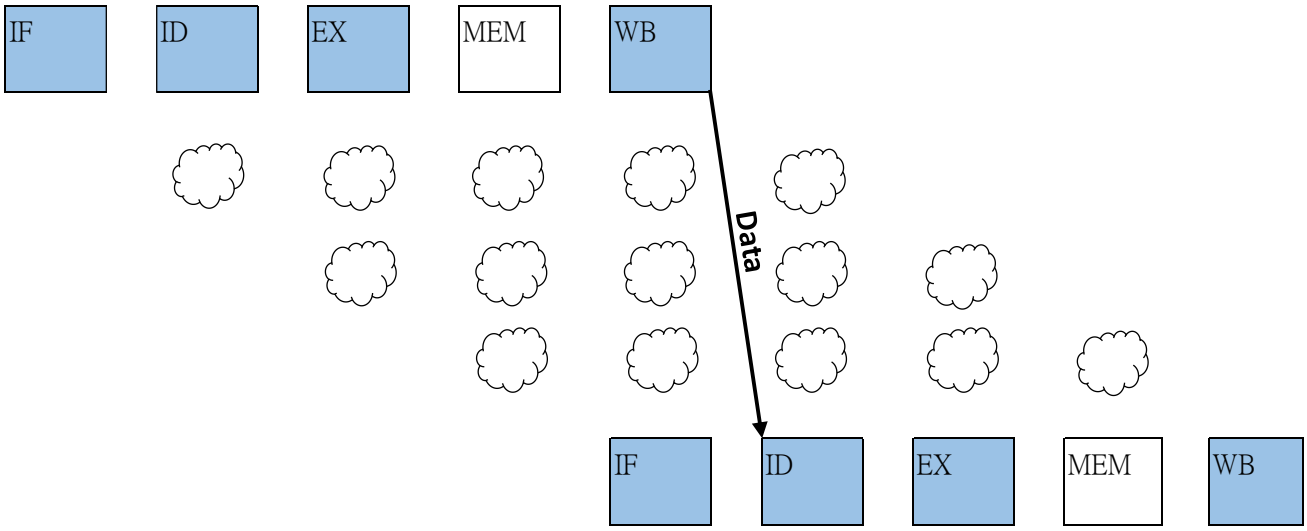
為了實現 pipeline，我們在五個 stage 都各設定一個能夠儲存 instruction 的變數，讓各 stage 執行完一個變數，得以將所執行的指令存至下個 stage 執行，而當前的 stage 則接收下一個指令，以此讓所有 stage 都能在正緣觸發各處理一個不同的指令，直到完成整個 ISA。

（我們的電路有一個部分與講義說明有所出入）

課本範例: (R-type)



我們的版本:



4. I/O of stages

Stage	Difinition	I/O	Parameter	Explanation
IF	計算下一個 pc 的值，並抓取對應的 instruction	input	branch	判斷是否 branch
			pc	給初始的 program counter
			jump	判斷是否 jump
			rst	初始化
			clk	時脈
		output	newpc	下個指令位置
			ins_IF	依據 newpc 產生的指令
ID	解碼 instruction，並對 I-type 的後 16bit 做 sign extend	input	ins_IF	依據 newpc 產生的指令
			zero	暫存器
			clk	時脈
			rst	初始化
		output	ins_ID	Pipeline 傳遞至 ID 的 instruction，為指令的第二個 stage
			sign_extend	將 I-type 最後的 16bits 補成 32bits
			Read_data1	rs 地址所對應的值
			Read_data2	rt 地址所對應的值
EX	對 I-type、R-type 做 ALU 運算	input	newpc	下個指令位置
			Read_data1	rs 地址所對應的值
			Read_data2	rt 地址所對應的值
			ins_ID	Pipeline 時 ID 輸入至 EX 的 instruction
			sign_extend	將 I-type 最後的 16bits 補成 32bits
			clk	時脈
			rst	初始化
		output	Alu_result	ALU 的運算結果
			ins_EX	Pipeline 傳遞至 EX 的 instruction，為指令的第三個 stage
			Write_data_mem	欲存入 memory 的值
			branch	判斷是否 branch
			jump	判斷是否 jump
MEM	當指令為 lw、sw 時，將 data 由 memory 讀出或寫入	input	ins_EX	Pipeline 時自 EX 輸入 MEM 的 instruction
			Alu_result	ALU 的運算結果
			Write_data_mem	欲存入 memory 的值
			clk	時脈

			rst	初始化
		output	Read_data_mem	自 memory 讀出，欲寫入 register 的值
			ins_MEM	Pipeline 傳遞至 MEM 的 instruction，為指令的第四個 stage
			Mem reg	memory
WB	將最後運算結果寫回暫存器	input	ins_MEM	Pipeline 時自 MEM 輸入 WB 的 instruction
			Read_data_mem	自 memory 讀出，欲寫入 register 的值
			clk	時脈
			rst	初始化
		output	zero	暫存器

5. Instruction Set

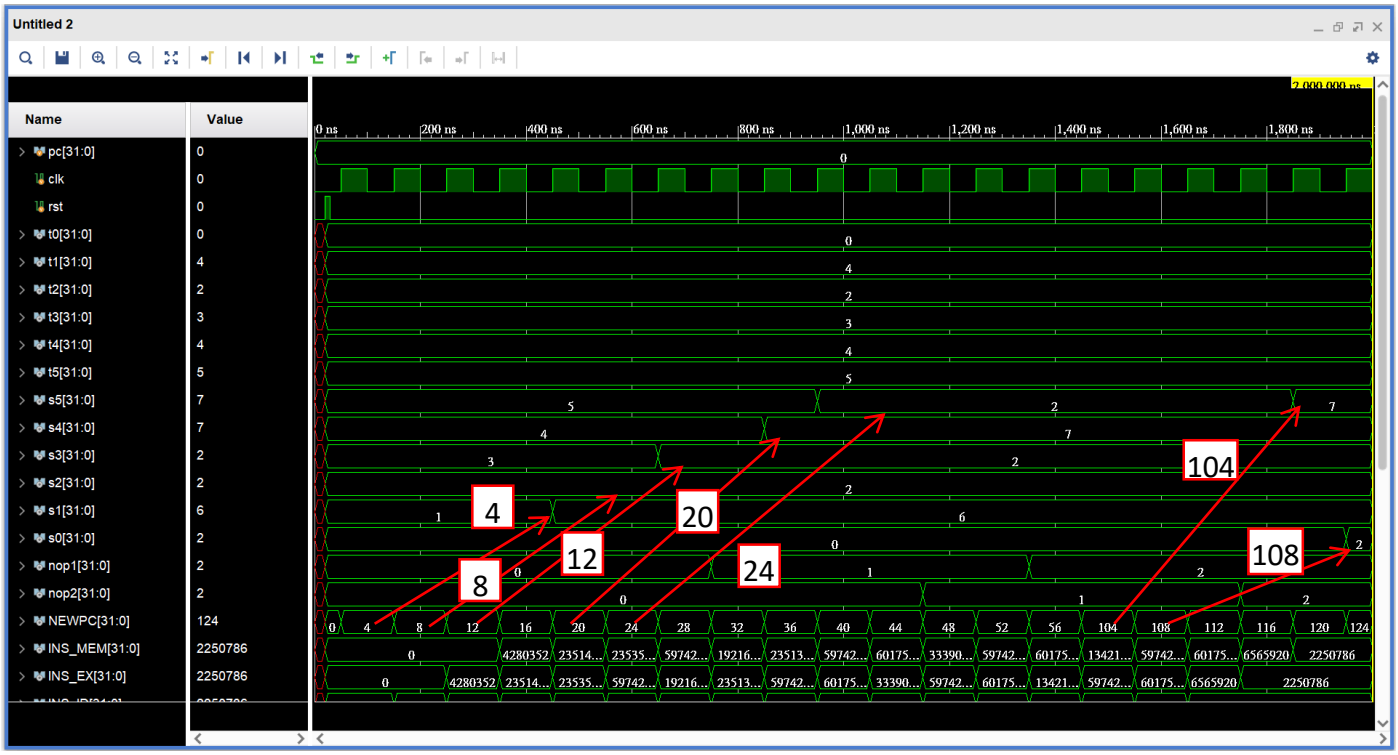
PC	Instruction	Machine Code	Result
4	add $s1, $r2, $r1	000000 00010 00001 01010 00000 100000	s1=6
8	lw $s2, 0($t1)	100011 00001 01001 00000 00000 000000	s2=2
12	lw $s3, 0($t2)	100011 00010 01000 00000 00000 000000	s3=2
16	nop	001000 11100 11100 00000 00000 000001	
20	LOOP1:or $s4, $t5, $s2	000000 01001 00101 00111 00000 100101	s4=7
24	lw $s5, 0($t1)	100011 00001 00110 00000 00000 000000	s5=2
28	nop	001000 11100 11100 00000 00000 000001	
32	nop	001000 11110 11110 00000 00000 000001	
36	beq $s5, 0 LOOP1	000100 11111 00110 11111 11111 111011	no branch
40	nop	001000 11100 11100 00000 00000 000001	
44	nop	001000 11110 11110 00000 00000 000001	
48	j LOOP2	000010 00000 00000 00000 00000 011010	nextpc = 104
52	nop	001000 11100 11100 00000 00000 000001	
56	nop	001000 11110 11110 00000 00000 000001	
104	LOOP2: add $s5, $r3, $r4	000000 00011 00100 00110 00000 100000	s5=7
108	sub $s0, $r1, $r2	000000 00001 00010 01011 00000 100010	s0=2

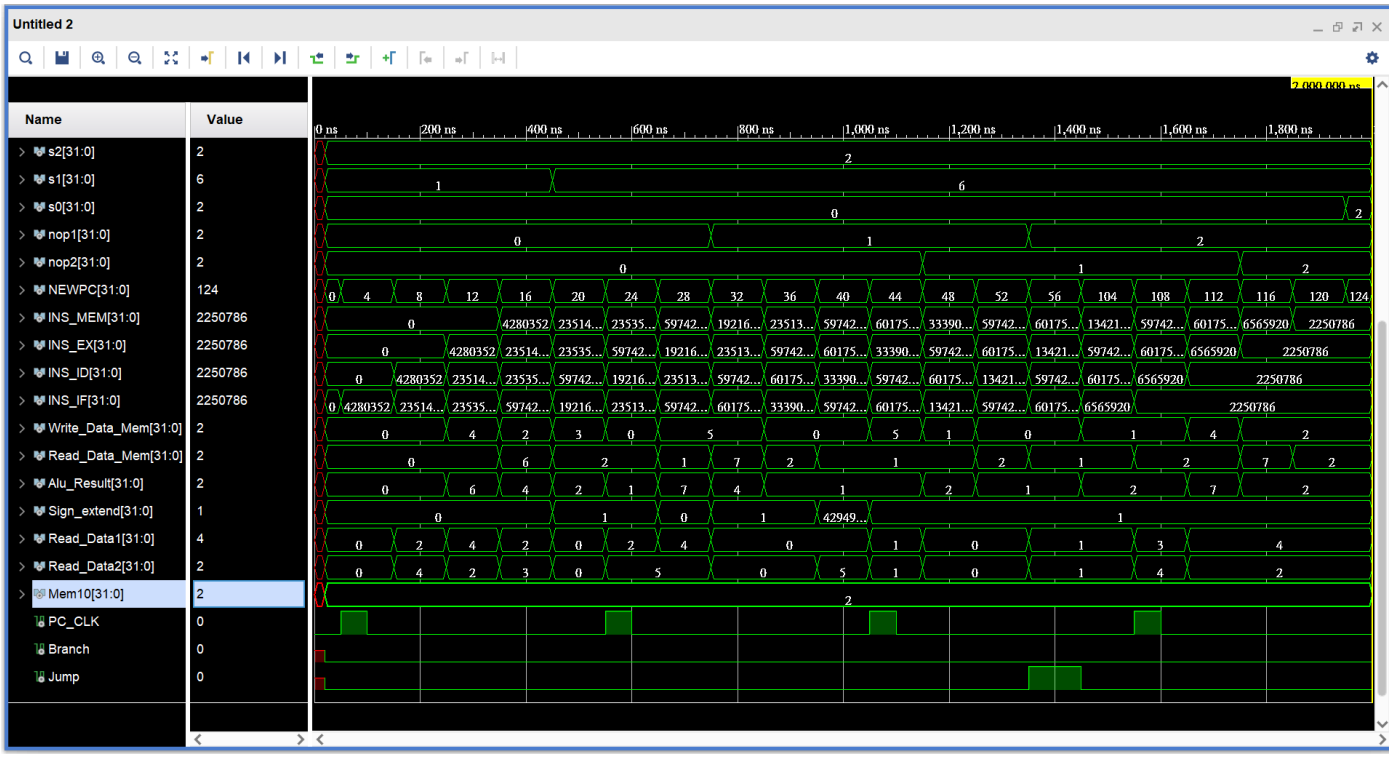
6. 指令解碼、暫存器及記憶體地址

function code		記憶體	記憶體位址(十進位)	值	暫存器	程式宣告名稱	地址	初始值
add	100000	mem0	40	2	t0	reg0	01100	0
sub	100010	mem1	36	2	t1	reg1	00001	4
and	100100	mem2	32	2	t2	reg2	00010	2
or	100101	mem3	28	2	t3	reg3	00011	3
slt	101010	mem4	24	2	t4	reg4	00100	4

op code		mem5	20	2	t5	reg5	00101	5
R-type	000000	mem6	16	2	s5	reg6	00110	5
lw	100011	mem7	12	2	s4	reg7	00111	4
sw	101011	mem8	8	2	s3	reg8	01000	3
beq	000100	mem9	4	2	s2	reg9	01001	2
addi	001000	mem10	0	2	s1	reg10	01010	1
					s0	reg11	01011	0
					nop1	reg12	11100	0
					nop2	reg13	11110	0
					zero	zero	11111	0

7. 波形圖



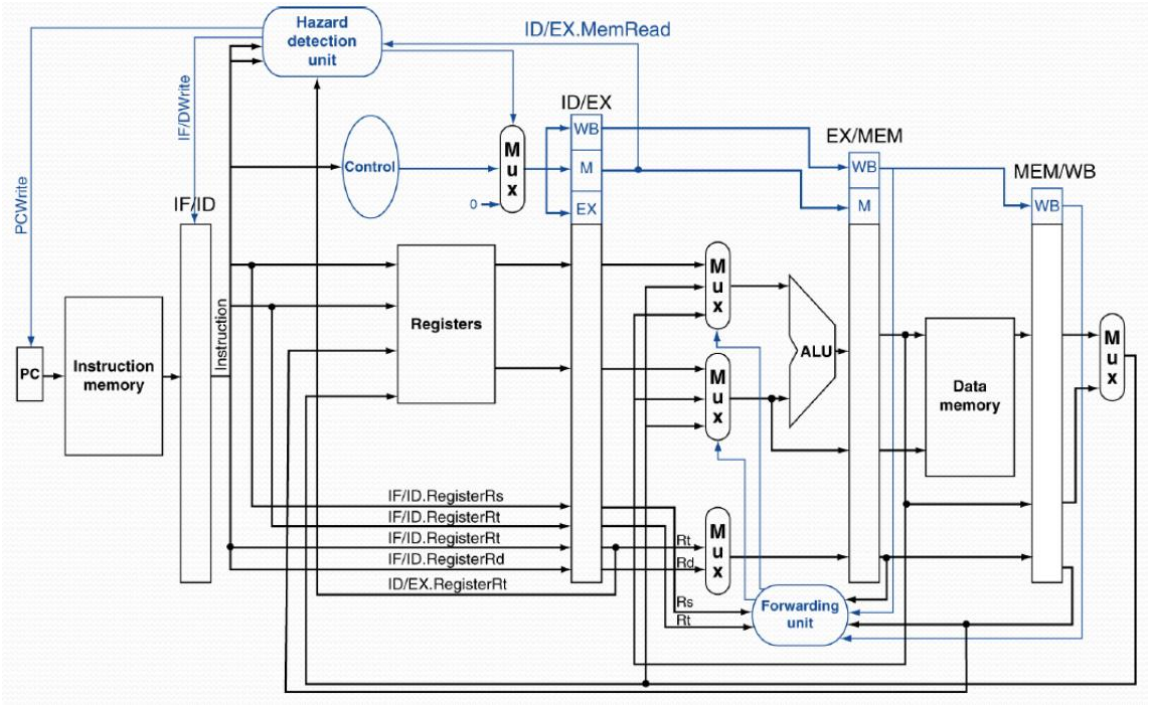


PC	Instruction	Result
4	add $s1, $r2, $r1	s1=6
8	lw $s2, 0($t1)	s2=2
12	lw $s3, 0($t2)	s3=2
16	nop	
20	LOOP1:or $s4, $t5, $s2	s4=7
24	lw $s5, 0($t1)	s5=2
28	nop	
32	nop	
36	beq $s5, 0 LOOP1	no branch
40	nop	
44	nop	
48	j LOOP2	nextpc = 104
52	nop	
56	nop	
104	LOOP2: add $s5, $r3, $r4	s5=7
108	sub $s0, $r1, $r2	s0=2

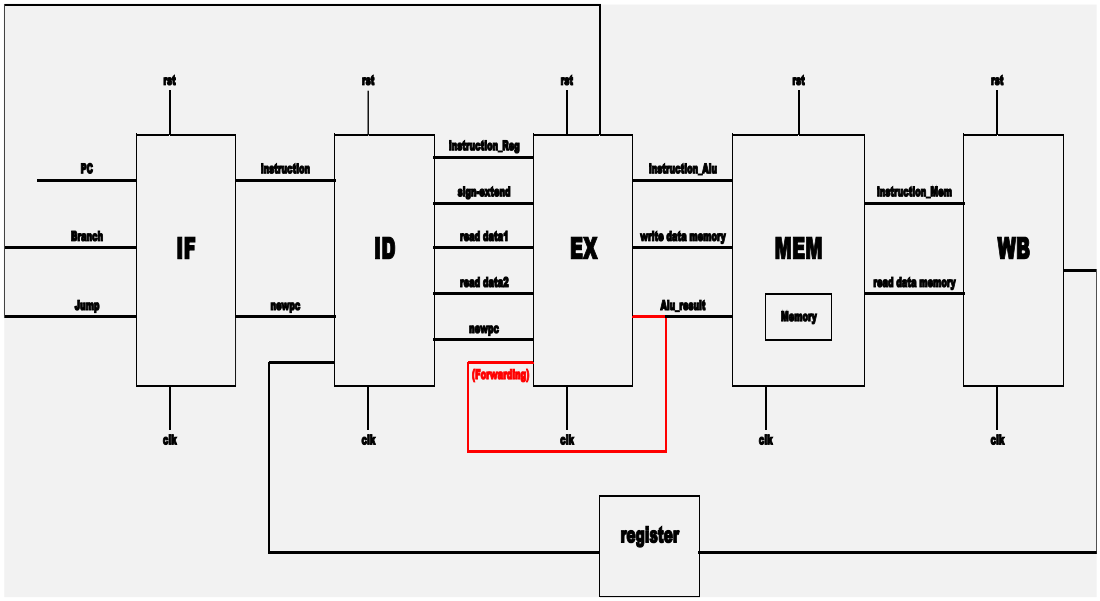
圖中的方框為 newpc 的值，各自對應不同的 instruction 以及運算結果，如左表所示。由於 pipeline 可能會有 hazard 的問題，因此在這裡我們加入 nop 解決。另外，由於 mem 中都是存 2 的關係，因此 lw 進去的值都會是 2。

三、 Hazard-Detection

1. 架構



2. Block Diagram



3. 架構說明

與 Pipelined 架構相同，將 EX 階段的運算結果(Alu_result)接回 EX 階段輸入，並設暫存器 verymissrabbit 讀取運算結果形成 Forwarding。

4. Instruction Set

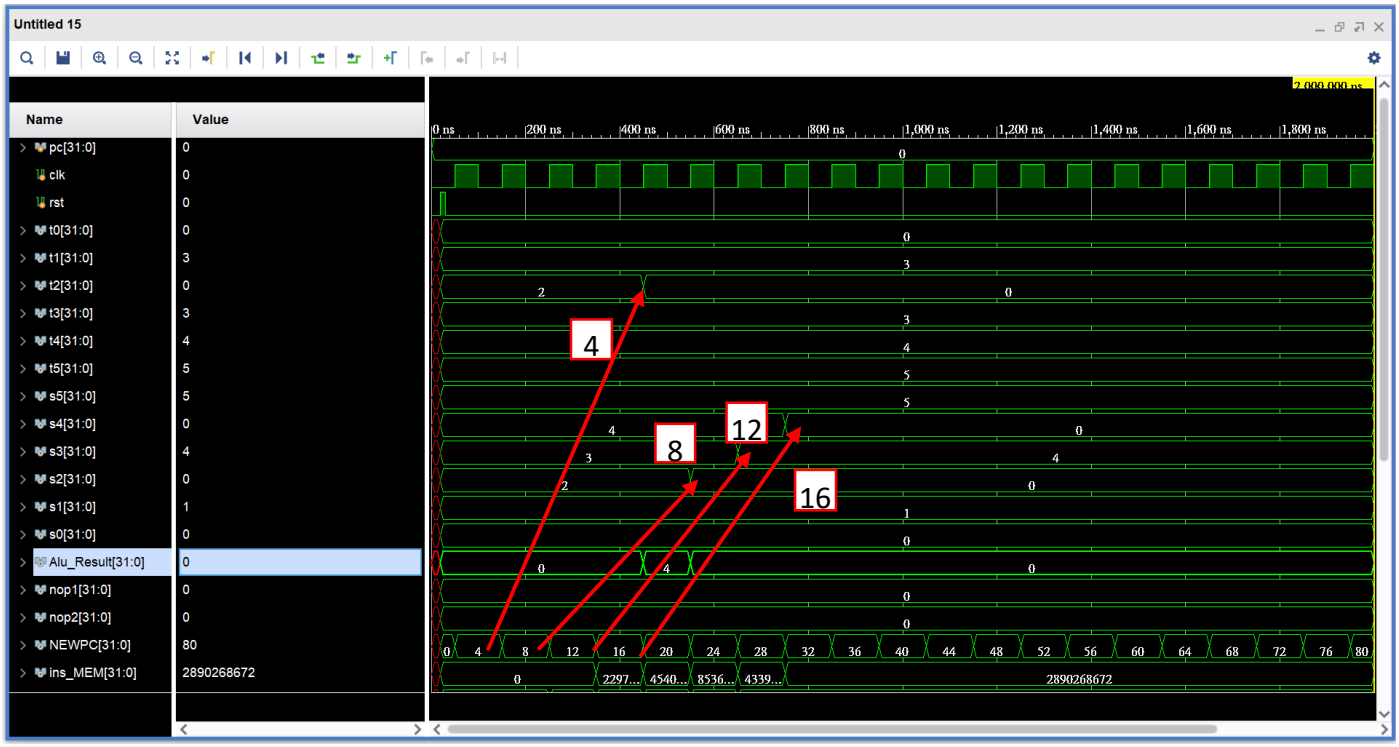
PC	Instruction	Machine Code	Result
4	sub $t2, $t1, $t3	000000 00001 00011 00010 00000 100010	t2 = 0
8	and $s2, $t2, $t5	000000 00010 00101 01001 00000 100100	s2 = 0
12	or $s3, $t4, $t2	000000 00100 00010 01000 00000 100101	s3 = 0

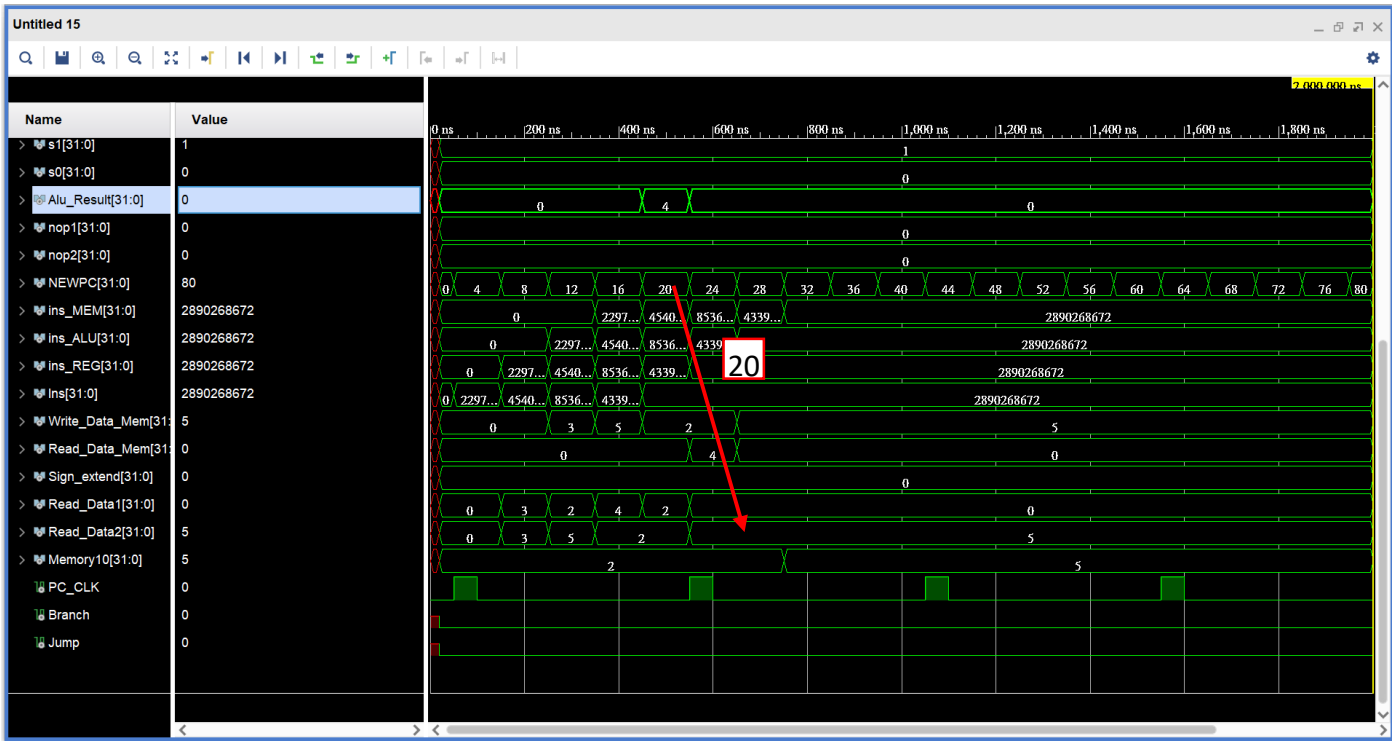
16	add $s4, $t2, $t2	000000 00010 00010 00111 00000 100000	s4 = 0
20	sw $s5, 0($t2)	101011 00010 00110 00000 00000 000000	mem10 = 5

5. 指令解碼、暫存器及記憶體地址

function code		記憶體	記憶體位址 (十進位)	值	暫存器	程式宣告名稱	地址	初始值
add	100000	mem0	40	2	t0	reg0	01100	0
sub	100010	mem1	36	2	t1	reg1	00001	4
and	100100	mem2	32	2	t2	reg2	00010	2
or	100101	mem3	28	2	t3	reg3	00011	3
slt	101010	mem4	24	2	t4	reg4	00100	4
op code		mem5	20	2	t5	reg5	00101	5
R-type	000000	mem6	16	2	s5	reg6	00110	5
lw	100011	mem7	12	2	s4	reg7	00111	4
sw	101011	mem8	8	2	s3	reg8	01000	3
beq	000100	mem9	4	2	s2	reg9	01001	2
addi	001000	mem10	0	2	s1	reg10	01010	1
					s0	reg11	01011	0
					nop1	reg12	11100	0
					nop2	reg13	11110	0
					zero	zero	11111	0

6. 波形圖





PC	Instruction	Result
4	sub $t2, $t1, $t3	t2 = 0
8	and $s2, $t2, $t5	s2 = 0
12	or $s3, $t4, $t2	s3 = 0
16	add $s4, $t2, $t2	s4 = 0
20	sw $s5, 0($t2)	mem10 = 5

圖中方框代表 newpc 的值，依照該 newpc 的值輸入指令並得到相對應的結果，如左表所示。
當 newpc=20 時，讀到的$s5 的值為 5，並將之寫入到 memory10 裡。

四、 工作分配

Sena	Zuri	Nori	Nova
25%	25%	25%	25%
設計架構/RTL code/Test bench code 書面報告(Word/PDF) 助教們辛苦了!!! Happy New Year~~~			